

Módulo 2

Programación Orientada a Objetos en C#



Colecciones

Curso de Iniciación a la programación con C#

Index

Colecciones.....	3
Listas.....	3
Novedades listas C#.....	5
Diccionarios (mapas).....	6
Conjuntos.....	8
Colecciones ordenadas.....	8

Colecciones

Además de los arrays estáticos, existen otras formas de agrupar objetos en diversos tipos de colecciones. Cada una con una serie de características útiles que serán más o menos adecuadas en función del contexto. Para ello se apoyan en una serie de clases e interfaces ya predefinidas.

Vamos a ver algunas de las principales colecciones genéricas. Es decir, trabajan con el tipo de datos que le indiquemos en el momento de la creación. El concepto de genéricos en programación orientada a objetos es bastante avanzado a la hora de implementar nosotros una clase que los utilice, pero es bastante sencillo de utilizar.

Este tipo de colecciones se encuentran definidas dentro del espacio de nombres **System.Collections.Generic**, por lo que conviene usarlo para no tener que escribirlo cada vez.

```
using System.Collections.Generic;
```

Listas

Las listas son bastante similares a los arrays estáticos, con la principal ventaja de que su tamaño se ajusta de manera dinámica a las necesidades de almacenamiento, además de tener ya implementados métodos como el borrado o la inserción en cualquier posición.

Para crear una lista, debemos crear un objeto de la clase `List` indicando el tipo de datos que va a almacenar de la siguiente manera:

```
// Lista que almacena números
var numeros = new List<int>();

// Podemos establecer una capacidad inicial reservada
var numeros = new List<int>(12);

// Se puede crear una lista a partir de un array u otra colección
int[] array = {2, 3, 5};
var numeros = new List<int>(array);
```

Si quisieramos almacenar cadenas, crearíamos una `List<string>`, para almacenar fechas `List<DateTime>`, así con cualquier tipo de datos. De hecho, podríamos hacer que se comportara como un array bidimensional si creamos una lista que contenga listas → `List<List<int>>`.

A los elementos de una lista accedemos de igual manera que si se tratara de un array, con la posición entre corchetes:

```
var numeros = new List<int>{4, 23, 6, 18, 9, 13};
numeros[1] = 24;
Console.WriteLine(numeros[1]); // 24
```

Algunos métodos y propiedades útiles para trabajar con listas son:

- **Add(item)** → Añade un item al final de la lista.

```

numeros.Add(3);
numeros.Add(5);
numeros.Add(6);

```

- **AddRange(colección)** → Añade todos los elementos de una colección (array, lista, etc.) al final de la lista.
- **Count** → Devuelve la cantidad de elementos de la lista (similar a la propiedad Length de los arrays).

```

Console.WriteLine(numeros.Count);

```

- **ToArray()** → Devuelve un tipo array estático a partir de la lista.

```

int[] numsArray = numeros.ToArray();

```

- **Clear()** → Borra todos los elementos de la lista.
- **Contains(item)** → Devuelve un booleano indicando si el elemento está en la lista o no.

```

Console.WriteLine(numeros.Contains(5)); // True

```

- **ConvertAll(función de transformación)** → Le aplica la función a todos los elementos de la lista y devuelve una nueva lista con los valores transformados.

```

// Multiplicamos todos los valores por 2
List<int> numeros2 = numeros.ConvertAll(n => n * 2);
Console.WriteLine(String.Join(", ", numeros2)); // 6,10,12

```

- **Find(función de condición)** → Busca y devuelve el primer elemento que cumpla la condición que devuelve la función recibida. Otras similares son **FindIndex** (devuelve el índice del primer elemento que cumple la condición), **FindLast** y **FindLastIndex** (como las anteriores pero devuelven el último elemento que cumple la condición), y **FindAll** (Devuelve una lista con todos los elementos que la cumplen).

```

// Lista que almacena números
List<int> numeros = new List<int>{12, 8, 23, 9, 3, 33, 7};
int numMenor = numeros.Find(n => n < 10); // 8
List<int> numMenores = numeros.FindAll(n => n < 10); // {8, 9, 3, 7}

```

- **ForEach(función de recorrido)** → Equivale al bucle foreach, pero en este caso recibe una función que aplica a cada elemento de la lista.

```

numeros.ForEach(n => Console.WriteLine(n));

```

- **GetRange(posición, cantidad)** → Devuelve una lista con una cantidad de elementos desde la posición indicada.

```

List<int> numeros = [12, 8, 23, 9, 3, 33, 7];
List<int> rango = numeros.GetRange(2, 4); // 23, 9, 3, 33

```

- **IndexOf(elemento)** → Igual que en un array, recibe un elemento y lo busca, devolviendo la primera posición donde lo encuentra o -1 si no está. **LastIndexOf** devuelve la última posición.
- **Insert(posición, elemento)** → Inserta el elemento en la posición indicada de la lista. Desplaza el resto de elementos a su derecha.
- **InsertRange(posición, colección)** → Añade los elementos de la colección recibida a partir de la posición indicada.

- **Remove(elemento)** → Busca y elimina el elemento de la lista. Devuelve un booleano indicando si lo ha encontrado (y borrado).
- **RemoveRange(posición, cantidad)** → Borra una cantidad de elementos a partir de la posición recibida.
- **RemoveAt(posición)** → Borra el elemento en la posición indicada.
- **RemoveAll(función de condición)** → Borra todos los elementos que cumplan la condición evaluada en la función recibida.

```
List<int> numeros = [4, 23, 6, 18, 9, 13];
numeros.RemoveAll(n => n % 2 == 0); // Borramos números pares
Console.WriteLine(String.Join(", ", numeros)); // 23, 9, 13
```

- **Reverse()** → Le da la vuelta a la lista.
- **Sort()** → Ordena la lista. Si no son elementos primitivos como números o cadenas, o queremos un orden especial, habría que pasarle una función de comparación (recibe 2 elementos y devuelve si el primero es menor -1, si son iguales 0, o si el primero es mayor 1).

```
var productos = new List<Producto>();
productos.Add(new Producto("Silla", 35));
productos.Add(new Producto("Mesa", 122));
productos.Add(new Producto("Ratón", 8.95));
productos.Add(new Producto("Teclado", 12.5));
productos.Sort((p1, p2) => p1.Precio.CompareTo(p2.Precio));
Console.WriteLine(String.Join("\n", productos));
```

```
Ratón: 8,95 euros
Teclado: 12,50 euros
Silla: 35,00 euros
Mesa: 122,00 euros
```

- **TrueForAll(función de condición)** → Recibe una función con una condición (predicado), que si todos los elementos cumplen devolverá true, y false en caso contrario.

Novedades listas C#

Todas las operaciones que hemos visto anteriormente que se pueden hacer con arrays, también se pueden hacer con listas, como por ejemplo las siguientes:

Rangos → Permiten generar una nueva lista a partir de una sección de otra lista. Esta operación sustituiría al método GetRange.

```
List<int> numeros = [12, 8, 23, 9, 3, 33, 7];
List<int> rango = numeros[2..5]; // 23, 9, 3, 33
```

Patrones de lista → permiten comparar si la lista sigue una estructura o patrón

```
List<int> numeros = [1, 2, 3, 4, 5];

// ¿La lista empieza por 1, luego tiene tres elementos cualesquiera (_) y termina en 5?
if (numeros is [1, _, _, _, 5]) { ... }

// ¿La lista empieza por 1, 2 y luego tiene cualquier cantidad de elementos (...)?
if (numeros is [1, 2, ..]) { ... }
```

Parámetros como listas → Recoger parámetros variables en un método como una lista en lugar de un array:

```
public void ProcesarElementos(params List<int> numeros) { ... }
```

Diccionarios (mapas)

Los diccionarios (también llamados mapas en otros lenguajes como Java o JavaScript) son colecciones del tipo **clave-valor**. Para referenciar los elementos de un array o una lista, utilizamos su posición, o lo que es lo mismo un índice o clave numérica.

En el caso de un diccionario, en lugar de un valor numérico (posición), se puede usar cualquier tipo de clave como puede ser una cadena o un objeto para hacer referencia a un valor. Al crear el diccionario, establecemos los tipos de la clave y el valor.

```
// Diccionario que almacena cadenas y el índice también es una cadena
var traducciones = new Dictionary<string, string>();
traducciones.Add("mesa", "table");
traducciones.Add("silla", "chair");
traducciones.Add("cabeza", "head");
traducciones.Add("botella", "bottle");
// [mesa, table],[silla, chair],[cabeza, head],[botella, bottle]
Console.WriteLine(String.Join(", ", traducciones));
```

Acceder a los valores

Para acceder al valor a partir de una clave, es el mismo concepto que en un array o lista. La diferencia es que el índice no tiene por qué ser numérico.

```
string palabra = traducciones["cabeza"];
Console.WriteLine(palabra); // head
```

Recorrer los valores de un diccionario

Las parejas de clave-valor de un diccionario se pueden recorrer con un bucle foreach. En este caso cada elemento de la colección se devuelve como de tipo **KeyValuePair<Tipo1, Tipo2>** (Los tipos son los mismos que en el diccionario).

```
foreach (var pair in traducciones)
{
    Console.WriteLine($"{pair.Key} en inglés es: {pair.Value}");
}
```

Otra opción sería recorrer directamente las claves (Keys) o valores (Values) del diccionario.

```
foreach (string key in traducciones.Keys)
{
    Console.WriteLine($"{key} en inglés es: {traducciones[key]}");
}

foreach (string value in traducciones.Values)
{
    Console.WriteLine($"Palabra: {value} (no puedo saber la clave...)");
}
```

Existe una tercera opción, que es utilizando un **iterador**. Es un concepto similar a leer las líneas de un archivo ya que vamos avanzando al siguiente elemento hasta que no podemos avanzar más. Todas los tipos de colecciones se pueden recorrer así.

```
var iterador = traducciones.GetEnumerator();
while(iterador.MoveNext())
{
    Console.WriteLine($"{iterador.Current.Key} en inglés es: {iterador.Current.Value}");
}
```

Algunos métodos útiles de los diccionarios:

- **Add(clave, valor)** → Añade una nueva pareja de clave y valor. Si la clave ya existe o es nula, devolverá un error.
- **ContainsKey(clave)** → Devuelve true o false si el diccionario contiene la clave. También existe **ContainsValue** para buscar valores.
- **GetValueOrDefault(clave, valor por defecto)** → Devuelve el valor asociado a una clave, y si esta no existe, te devuelve el valor especificado por defecto.
- **Remove(clave)** → Borra la pareja clave-valor asociada a la clave.

Como ya hemos dicho, tanto la clave como el valor pueden ser cualquier cosa:

```
class Tarea
{
    private string descripcion;
    private bool acabada;

    public Tarea(string descripcion)
    {
        this.Descripcion = descripcion;
        this.Acabada = false;
    }

    public string Descripcion { get => descripcion; set => descripcion = value; }
    public bool Acabada { get => acabada; set => acabada = value; }

    public override string ToString()
    {
        return Descripcion + " (" + (Acabada? "finalizada" : "pendiente") + ")";
    }
}

var agenda = new Dictionary<DateTime, Tarea>();
agenda.Add(DateTime.Now.AddDays(3), new Tarea("Ir a comprar"));
agenda.Add(DateTime.Parse("12/10/2019"), new Tarea("Cumpleaños de mi mapache"));
agenda.Add(DateTime.Now.AddHours(5), new Tarea("Sacar la basura"));

foreach(var pair in agenda)
{
    Console.WriteLine($"{pair.Key} -> {pair.Value}");
}

28/7/19 12:18:28 -> Ir a comprar (pendiente)
12/10/19 0:00:00 -> Cumpleaños de mi mapache (pendiente)
25/7/19 17:18:28 -> Sacar la basura (pendiente)
```

Conjuntos

Los conjuntos se podrían entender como diccionarios pero solo almacenando claves, sin valor asociado. Este tipo de colección almacena un conjunto de valores, y no admite ningún valor repetido. Es decir, un valor puede estar o no estar en el conjunto. No existe el concepto de la posición tampoco.

```
HashSet<string> set = new HashSet<string>();
set.Add("Manzana");
set.Add("Plátano");
set.Add("Calabaza");
set.Add("Manzana");
set.Add("Plátano");

Console.WriteLine(String.Join(", ", set)); // Manzana,Plátano,Calabaza

Console.WriteLine(set.Contains("Manzana")); // True

set.Remove("Manzana");
Console.WriteLine(String.Join(", ", set)); // Plátano,Calabaza
```

Colecciones ordenadas

Por defecto, al recorrer un diccionario o un conjunto, se nos devuelven los valores en el orden que se insertaron. Existe la posibilidad de crear colecciones ordenadas (SortedDictionary, SortedSet), que mantienen un orden interno al insertar y borrar, y cuando las recorremos seguirán dicho orden.

El orden se establece mediante una función de ordenación que se le pasa por parámetro al constructor. Si no se le pasa ninguna, se establece el orden primitivo de las claves (orden numérico ascendente, orden alfabético), pero en el caso de que la clave sea un objeto, conviene establecer dicha ordenación.

```
// Al ser la clave string, por defecto se ordenan alfabéticamente
var traducciones = new SortedDictionary<string, string>();
traducciones.Add("mesa", "table");
traducciones.Add("silla", "chair");
traducciones.Add("cabeza", "head");
traducciones.Add("botella", "bottle");

foreach(KeyValuePair<string, string> pair in traducciones)
{
    Console.WriteLine($"{pair.Key} en inglés es: {pair.Value}");
}

botella en inglés es: bottle
cabeza en inglés es: head
mesa en inglés es: table
silla en inglés es: chair

// Creamos un conjunto de figuras ordenadas por área
var comparador = Comparer<IFigura>.Create((f1, f2) =>
f1.GetArea().CompareTo(f2.GetArea()));
var figuras = new SortedSet<IFigura>(comparador);
figuras.Add(new Circulo(12));
figuras.Add(new Cuadrado(5));
figuras.Add(new Circulo(4));
figuras.Add(new Cuadrado(7.5));

foreach(IFigura figura in figuras)
{
    Console.WriteLine($"{figura.GetType()} -> Área: {figura.GetArea():F2}");
}
```

```
Mi_Proyecto.Cuadrado -> Área: 25,00  
Mi_Proyecto.Circulo -> Área: 50,27  
Mi_Proyecto.Cuadrado -> Área: 56,25  
Mi_Proyecto.Circulo -> Área: 452,39
```